EXPERIENTIAL LEARNING • APACHE SPARK

E-commerce Sales Performance Report

A junior-analyst briefing for management, built from 5,000 order records using Spark DataFrames and Spark SQL.

5,000	₹9.28 Cr	₹2.76 Cr	29.76%	3.96
Total orders	Total revenue	Total profit	Blended margin	Avg. rating

Scenario. Management asked for a quick sales-performance view of the 2025 order book. The workbook *ecommerce_sales_5000.xlsx* was converted to CSV and loaded into a local Spark session. Aggregations use the DataFrame API (`groupBy`, `agg`, `withColumn`). One core briefing is then rebuilt with Spark SQL to show the same numbers from a temporary view.

Coverage. Orders run from 2025-01-01 to 2025-12-31 across 8 cities, 6 categories, 24 products and 2,150 customers. There are no missing values. 845 orders have zero revenue — all of them are Cancelled.

1. Load and inspect the dataset

The Excel file was saved as CSV so Spark could infer types without an Excel connector. A local session then read the file with a header row and a date conversion on `order_date`.

```
spark = SparkSession.builder.master("local[*]").appName("EcommerceSales").getOrCreate()
orders = (spark.read.option("header", True).option("inferSchema", True)
               .csv("data/ecommerce_sales_5000.csv")
               .withColumn("order_date", to_date(col("order_date"))))
orders.printSchema()
orders.show(5, truncate=False)
print(orders.count(), len(orders.columns))
```

#	Column	Type	#	Column	Type
1	order_id	string	11	discount_pct	int
2	order_date	date	12	revenue	double
3	customer_id	string	13	cost	double
4	customer_segment	string	14	profit	double
5	city	string	15	payment_method	string
6	state	string	16	sales_channel	string
7	category	string	17	customer_age	int
8	product	string	18	customer_rating	int
9	quantity	int	19	order_status	string
10	unit_price	double			

Figure: inferred Spark schema after CSV load (19 columns, 5,000 rows).

Order ID	Date	City	Category	Product	Revenue	Status
ORD100001	2025-03-24	Kolkata	Fashion	Jeans	₹20,968.10	Delivered
ORD100002	2025-05-22	Delhi	Home & Kitchen	Mixer	₹8,050.31	Delivered
ORD100003	2025-07-28	Hyderabad	Beauty	Moisturizer	₹67,032.08	Returned
ORD100004	2025-08-03	Kolkata	Fashion	Jeans	₹70,750.83	Delivered
ORD100005	2025-11-06	Bengaluru	Electronics	Smartphone	₹20,579.28	Delivered
ORD100006	2025-09-12	Ahmedabad	Home & Kitchen	Bedsheet	₹17,829.71	Delivered
ORD100007	2025-07-07	Hyderabad	Beauty	Moisturizer	₹75,811.29	Delivered
ORD100008	2025-09-04	Hyderabad	Electronics	Laptop	₹15,339.92	Delivered

Table: first eight rows after ordering by order_id (inspection sample).

Order status	Orders	Share of book	Revenue	Profit
Delivered	3,310	66.20%	₹74,471,283.08	₹22,022,861.81
Returned	845	16.90%	₹18,322,131.06	₹5,588,434.00
Cancelled	845	16.90%	₹0.00	₹0.00

Quality note. Cancelled lines are booked at ₹0. Returned lines still carry revenue and profit, so a ‘realized sales’ view would need an extra filter on order_status.

2. Total number of orders

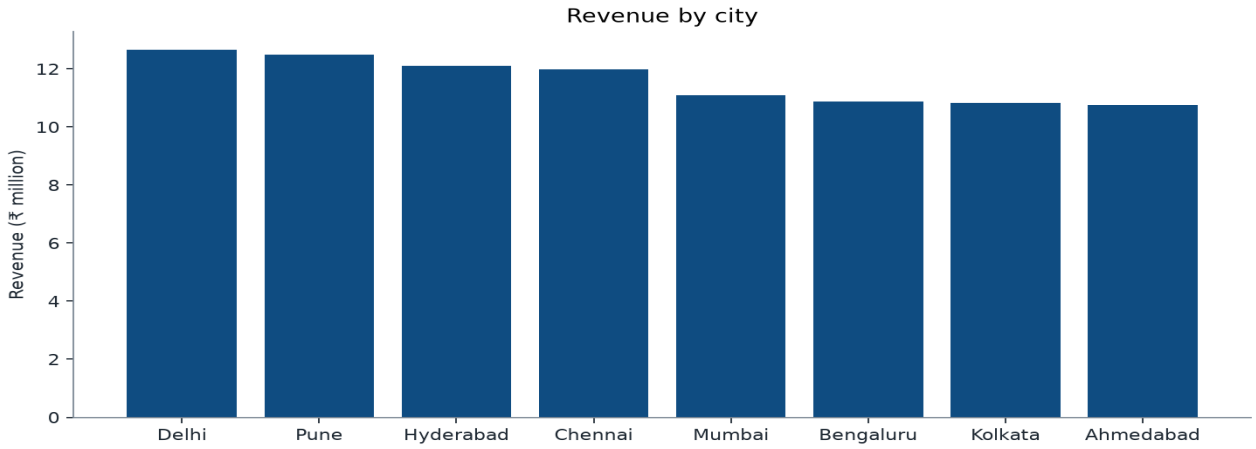
There are 5,000 orders in the file — one unique `order_id` per row, so a simple `orders.count()` is the order count. Of these, 3,310 were delivered (66.20%), 845 returned and 845 cancelled.

3. Total revenue and total profit by city

City totals are a grouped sum of revenue and profit. A safe blended margin (profit ÷ revenue) is attached only when the city revenue is positive.

```
orders.groupBy("city").agg(
    count("*").alias("orders"),
    round(sum("revenue"), 2).alias("total_revenue"),
    round(sum("profit"), 2).alias("total_profit")
).orderBy(col("total_revenue").desc())
```

City	Orders	Total revenue	Total profit	Margin
Delhi	657	₹12,650,984.52	₹3,817,733.22	30.18%
Pune	644	₹12,484,158.98	₹3,701,772.50	29.65%
Hyderabad	651	₹12,101,931.71	₹3,633,694.86	30.03%
Chennai	628	₹11,995,339.96	₹3,549,486.89	29.59%
Mumbai	612	₹11,095,836.74	₹3,344,799.09	30.14%
Bengaluru	608	₹10,877,062.27	₹3,173,110.79	29.17%
Kolkata	601	₹10,833,968.88	₹3,219,358.45	29.72%
Ahmedabad	599	₹10,754,131.08	₹3,171,340.01	29.49%

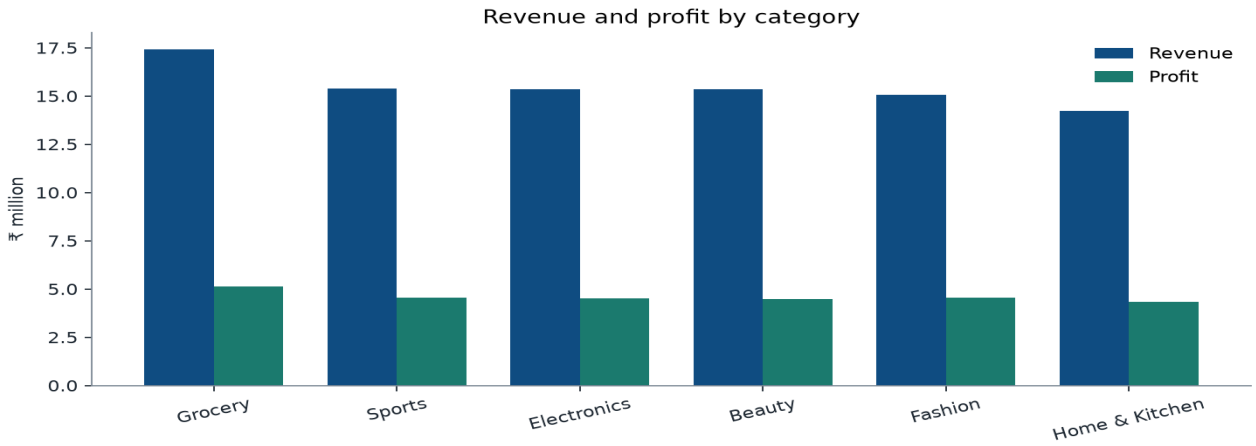


Delhi leads on revenue (₹12,650,984.52), followed by Pune and Hyderabad. The eight-city spread is tight: Ahmedabad at the bottom is still about 85% of Delhi's revenue, so the book is national rather than metro-concentrated.

4. Orders, revenue, profit and average rating by category

Category is the natural merchandise cut. Rating is averaged from `customer_rating` (1–5).

Category	Orders	Revenue	Profit	Avg rating	Margin
Grocery	868	₹17,414,860.34	₹5,147,519.64	4.00	29.56%
Sports	829	₹15,376,130.02	₹4,549,301.88	3.96	29.59%
Electronics	817	₹15,365,081.25	₹4,535,543.62	3.91	29.52%
Beauty	850	₹15,337,414.39	₹4,481,837.72	3.93	29.22%
Fashion	811	₹15,063,862.08	₹4,570,969.32	3.95	30.34%
Home & Kitchen	825	₹14,236,066.06	₹4,326,123.63	3.99	30.39%

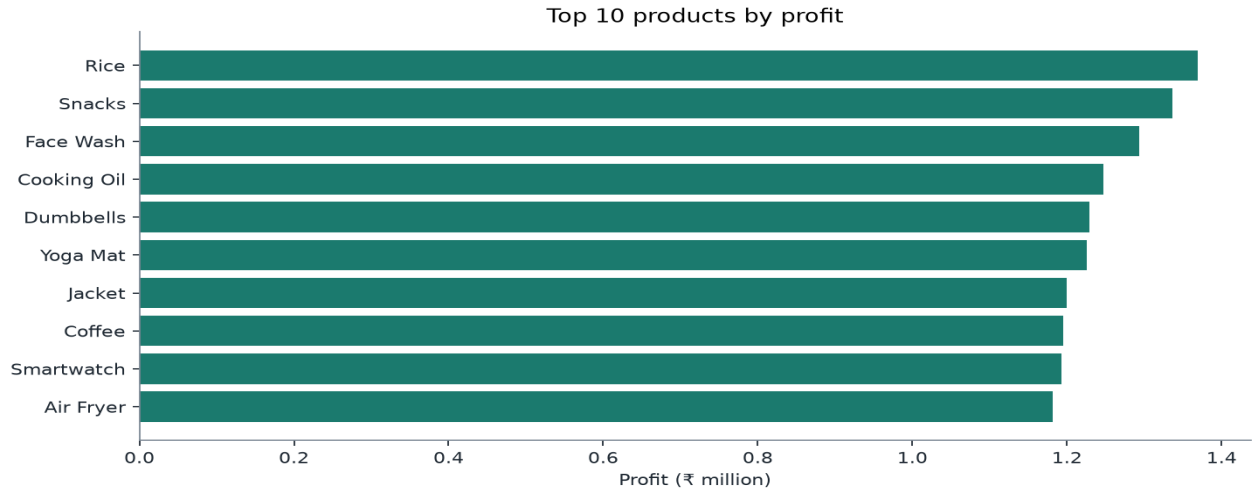


Grocery is the largest category by revenue (₹17,414,860.34, 868 orders). Average ratings sit in a narrow band around 4.0, so rating does not currently separate winners from the rest.

5. Top 10 products by profit

Products are ranked on summed profit, not unit price. That answers ‘which SKUs actually make money’ rather than which ones simply sell at a high list price.

Rank	Product	Category	Orders	Revenue	Profit	Margin
1	Rice	Grocery	221	₹4,801,709.09	₹1,369,626.23	28.52%
2	Snacks	Grocery	219	₹4,470,761.35	₹1,336,204.66	29.89%
3	Face Wash	Beauty	238	₹4,478,414.42	₹1,293,784.69	28.89%
4	Cooking Oil	Grocery	212	₹4,072,749.37	₹1,246,921.30	30.62%
5	Dumbbells	Sports	187	₹4,093,671.67	₹1,229,379.27	30.03%
6	Yoga Mat	Sports	221	₹4,003,845.77	₹1,225,124.64	30.60%
7	Jacket	Fashion	200	₹3,820,553.82	₹1,199,661.85	31.40%
8	Coffee	Grocery	216	₹4,069,640.53	₹1,194,767.45	29.36%
9	Smartwatch	Electronics	208	₹3,939,913.22	₹1,192,306.87	30.26%
10	Air Fryer	Home & Kitchen	219	₹3,841,249.10	₹1,181,840.42	30.77%



Rice (Grocery) is the profit leader at ₹1,369,626.23. Grocery staples and a few high-velocity Beauty and Sports SKUs fill most of the top ten.

6. Best city / category combinations by revenue

Crossing city with category shows where the catalog actually converts, which is more useful for inventory and campaign planning than either dimension alone.

```
orders.groupBy("city", "category").agg(
    count("*").alias("orders"),
    round(sum("revenue"), 2).alias("total_revenue"),
    round(sum("profit"), 2).alias("total_profit")
).orderBy(col("total_revenue").desc()).limit(15)
```

City	Category	Orders	Revenue	Profit
Delhi	Sports	119	₹2,574,230.03	₹749,621.90
Pune	Electronics	116	₹2,365,000.04	₹712,189.23
Hyderabad	Grocery	115	₹2,324,288.04	₹662,903.20
Pune	Grocery	104	₹2,291,914.65	₹655,921.74
Mumbai	Grocery	116	₹2,267,679.46	₹677,369.32
Delhi	Beauty	108	₹2,257,162.31	₹684,902.78
Kolkata	Grocery	105	₹2,252,838.57	₹665,931.96
Hyderabad	Fashion	110	₹2,240,459.11	₹657,859.50
Hyderabad	Sports	108	₹2,167,386.67	₹653,729.70
Chennai	Fashion	98	₹2,160,978.43	₹632,954.37
Delhi	Grocery	109	₹2,152,645.89	₹674,253.57
Bengaluru	Electronics	108	₹2,135,419.63	₹580,347.34
Ahmedabad	Grocery	102	₹2,082,921.80	₹645,112.90
Chennai	Grocery	110	₹2,080,492.87	₹584,899.44
Pune	Home & Kitchen	100	₹2,070,878.73	₹614,391.61

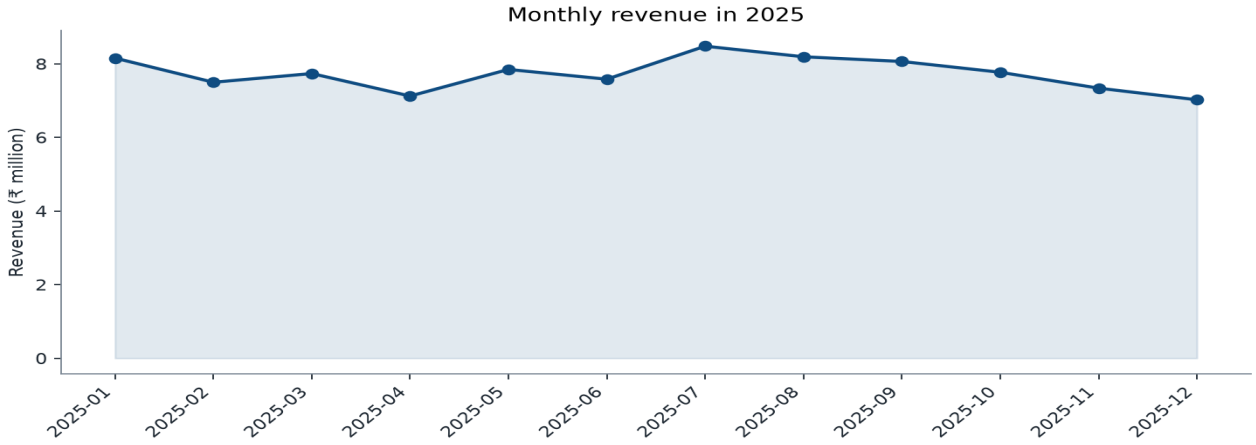
The strongest pair is Delhi × Sports (₹2,574,230.03 revenue, 119 orders).

7. Monthly revenue

A year-month key is derived with `date_format(order_date, 'yyyy-MM')` and revenue is summed. The series covers every calendar month of 2025.

Month	Orders	Revenue	Profit
2025-01	425	₹8,153,892.08	₹2,392,218.80
2025-02	390	₹7,497,179.32	₹2,265,106.18
2025-03	412	₹7,734,323.99	₹2,227,848.48
2025-04	427	₹7,123,070.37	₹2,189,963.92
2025-05	413	₹7,846,031.39	₹2,368,762.24
2025-06	413	₹7,580,733.25	₹2,259,357.90
2025-07	460	₹8,477,839.31	₹2,511,931.19
2025-08	433	₹8,190,037.68	₹2,452,910.27
2025-09	440	₹8,061,999.21	₹2,395,972.40

Month	Orders	Revenue	Profit
2025-10	403	₹7,769,394.37	₹2,305,391.36
2025-11	399	₹7,337,087.35	₹2,142,355.60
2025-12	385	₹7,021,825.82	₹2,099,477.47



Revenue peaks in 2025-07 at ₹8,477,839.31 and is quietest in 2025-12 (₹7,021,825.82). The year is commercially active throughout; there is no collapsed off-season.

8. Profit-margin column, created safely

A row-level margin of profit ÷ revenue is useful, but 845 cancelled orders have revenue = 0. Dividing blindly would throw Infinity or crash a job. The column is therefore wrapped in `when / otherwise`: null revenue or revenue ≤ 0 becomes a SQL null, not a number. The same idea at aggregate level uses `NULLIF(SUM(revenue), 0)`.

```
orders = orders.withColumn(
    "profit_margin",
    when(col("revenue").isNull() | (col("revenue") <= 0), lit(None).cast("double"))
    .otherwise(round(col("profit") / col("revenue"), 4))
)
```

Spark stored a null margin on 845 rows — matching the cancelled zero-revenue population. Delivered and returned rows keep a finite ratio.

order_id	Status	City	Product	Revenue	Profit	Margin
ORD100722	Delivered	Bengaluru	Air Fryer	₹39,329.77	₹17,697.83	45.00%
ORD104785	Returned	Hyderabad	Bedsheet	₹39,512.10	₹17,779.14	45.00%
ORD100709	Returned	Ahmedabad	Mixer	₹8,237.53	₹3,706.13	44.99%
ORD101390	Delivered	Pune	Coffee	₹6,433.21	₹2,893.96	44.98%
ORD101515	Delivered	Delhi	Face Wash	₹62,366.25	₹28,046.25	44.97%
ORD103621	Returned	Chennai	Bedsheet	₹17,004.02	₹7,644.43	44.96%
ORD104256	Returned	Delhi	Mixer	₹12,913.02	₹5,805.24	44.96%
ORD102885	Delivered	Ahmedabad	Rice	₹46,058.26	₹20,702.66	44.95%

Table: eight highest finite profit-margin orders after the safe column is added.

9. Repeat an important report in Spark SQL

The category briefing from Task 4 is the report management is most likely to reuse, so it is rebuilt from a temporary view. The SQL uses `NULLIF` so a zero-revenue group cannot divide by zero — the same safety rule as Task 8, written in SQL.

```
SELECT
    category,
    COUNT(*) AS total_orders,
    ROUND(SUM(revenue), 2) AS total_revenue,
    ROUND(SUM(profit), 2) AS total_profit,
    ROUND(AVG(customer_rating), 2) AS avg_rating,
    ROUND(SUM(profit) / NULLIF(SUM(revenue), 0), 4) AS profit_margin
FROM orders
GROUP BY category
ORDER BY total_revenue DESC
```

Category	Orders	Revenue	Profit	Avg rating	Margin
Grocery	868	₹17,414,860.34	₹5,147,519.64	4.00	29.56%
Sports	829	₹15,376,130.02	₹4,549,301.88	3.96	29.59%
Electronics	817	₹15,365,081.25	₹4,535,543.62	3.91	29.52%
Beauty	850	₹15,337,414.39	₹4,481,837.72	3.93	29.22%
Fashion	811	₹15,063,862.08	₹4,570,969.32	3.95	30.34%
Home & Kitchen	825	₹14,236,066.06	₹4,326,123.63	3.99	30.39%

The Spark SQL table matches the DataFrame category table. That is the point of the exercise: the same business question, two Spark APIs, identical totals.

10. What the data suggests

The 5,000-order book generated ₹92,793,414.14 in revenue and ₹27,611,295.81 in profit, a blended margin of 29.76%. That is a commercially healthy year on paper, but it is not fully collected cash: 845 cancelled orders (16.90%) contribute nothing, and another 845 returned orders still sit in the ledger with ₹18,322,131.06 of revenue attached.

Demand is geographically balanced. Delhi is the revenue leader, yet the gap to the smallest city is modest, so the catalog is not dependent on one metro. Category mix is similarly even; Grocery leads, but no vertical is a rounding error. The profit ranking is more concentrated at SKU level: Rice, Snacks, Face Wash and Cooking Oil should be protected on availability and discounting because they do the heavy lifting on profit.

The best city–category pair is Delhi / Sports. That is a practical place to test bundles, paid acquisition and warehouse depth. Monthly revenue stays in a workable band, peaking in 2025-07 and dipping in 2025-12, so planning can assume a steady run-rate rather than a single festival spike.

Customer rating averages about 4.0 in every category and therefore does not currently tell management where quality is failing. The sharper operational levers are reducing cancellations (zero-revenue leakage), deciding whether returned GMV belongs in the headline P&L, and keeping the top-profit grocery, beauty and sports SKUs in stock in Delhi, Pune and Hyderabad.

How to reproduce

From the `spark_experiential_learning` folder, with Java and PySpark installed:

```
python3 spark_sales_analysis.py
```

The script writes this PDF, plus chart PNGs under `charts/`. Numbers are computed in Spark; matplotlib is used only to draw the figures that go into the PDF.